

CONTROL DE ASISTENCIA DEL CURSO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicite el número de estudiantes y el número de días. Para cada estudiante registre su asistencia mediante P (presente) o A (ausente). Al finalizar muestre las asistencias y ausencias de cada estudiante y los totales del curso.

Estructura sugerida: for anidado. Conceptos: ciclos anidados, contadores y validación.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. estudiantes: número entero mayor que cero. Se repite la lectura si es cero o negativo.
2. días: número entero mayor que cero, común para todos los estudiantes. Se repite la lectura si no es positivo.
3. marca: exactamente P para presente o A para ausente, en mayúsculas. Cualquier otro texto se rechaza y se vuelve a pedir para el mismo día.
4. Las cantidades se ingresan como números enteros, sin letras ni decimales. Cada estudiante se identifica por su número, desde 1 hasta estudiantes.

1.2 Proceso

5. Inicializar totalAsistencias y totalAusencias en cero. Leer y validar estudiantes y días con while.
6. Crear dos arreglos de contadores: asistencias y ausencias, con una posición por estudiante. Permiten conservar los resultados hasta finalizar el registro.
7. Recorrer los estudiantes con el for externo. Inicializar sus dos contadores en cero y recorrer sus días con el for interno.
8. Leer marca y repetir con while mientras sea distinta de P y A. Si es P, aumentar asistencias[i] y totalAsistencias; si es A, aumentar ausencias[i] y totalAusencias.
9. Una marca inválida no incrementa contadores ni avanza al siguiente día. Completar todos los días de un estudiante antes de pasar al siguiente.
10. Al terminar ambos ciclos, recorrer los arreglos para mostrar los resultados individuales y después mostrar los totales del curso.

1.3 Salida de datos

11. Solicitudes que indican el estudiante y el día, y avisos de cantidades o marcas inválidas.
12. Cantidad de asistencias y ausencias de cada estudiante al finalizar el registro.
13. Total de asistencias y total de ausencias del curso. Su suma debe ser estudiantes × días.

2. ALGORITMO

14. Inicio.

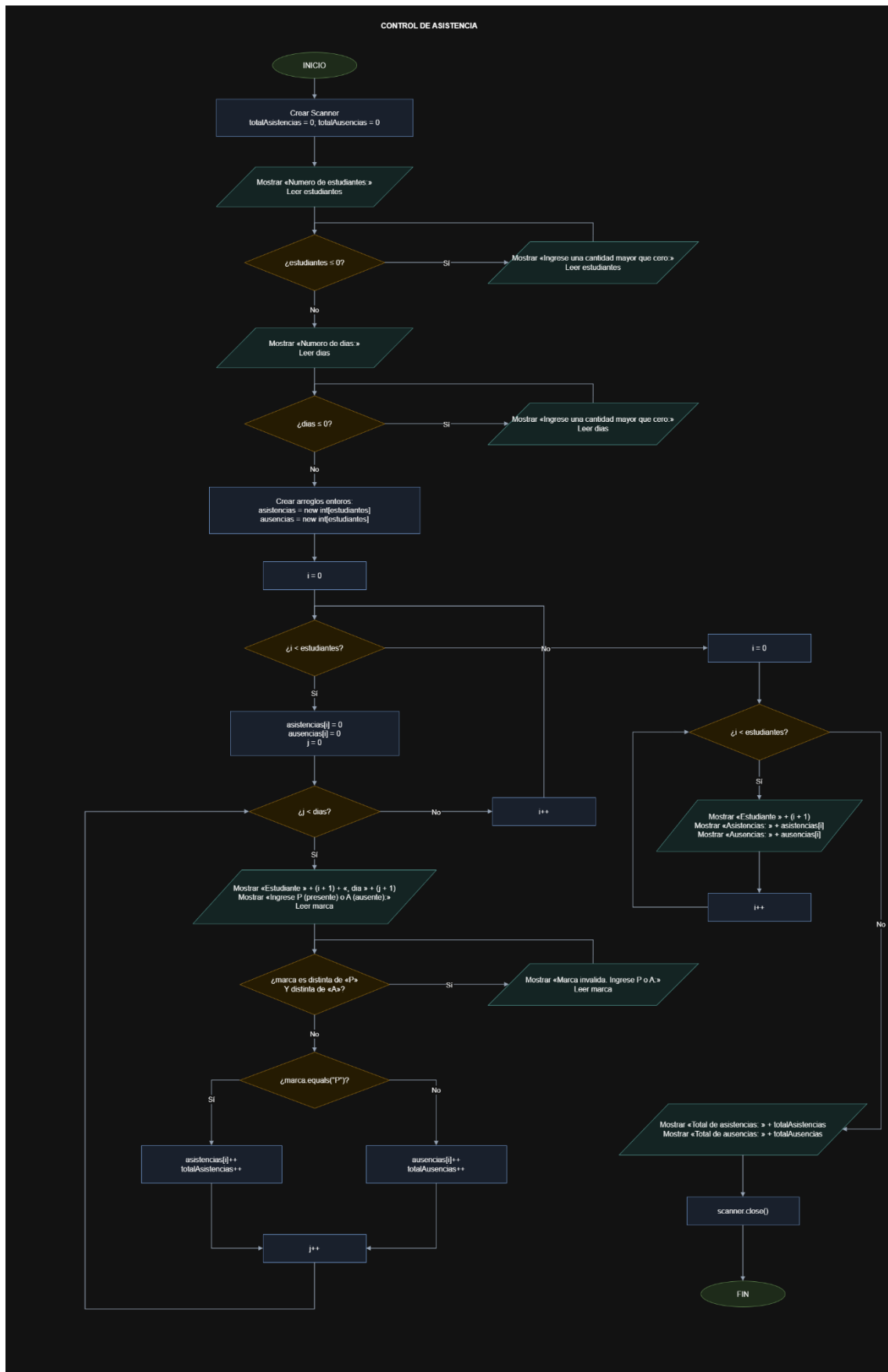
15. Declarar las variables e inicializar los totales de asistencias y ausencias en cero.
16. Solicitar el número de estudiantes; repetir la lectura mientras no sea positivo.
17. Solicitar el número de días; repetir la lectura mientras no sea positivo.
18. Crear dos arreglos de tamaño igual al número de estudiantes para guardar sus asistencias y ausencias.
19. Recorrer a los estudiantes con el ciclo externo y poner sus dos contadores en cero.
20. Para cada estudiante, recorrer todos los días con un ciclo interno.
21. Mostrar el número del estudiante y del día; solicitar la marca de asistencia.
22. Mientras la marca no sea P ni A, mostrar un aviso y solicitarla nuevamente para ese mismo día.
23. Si la marca es P, incrementar las asistencias del estudiante y el total de asistencias.
En caso contrario, incrementar sus ausencias y el total de ausencias.
24. Avanzar al siguiente día. Después de completar los días, continuar con el siguiente estudiante.
25. Al terminar el registro completo, recorrer los estudiantes y mostrar sus asistencias y ausencias.
26. Mostrar los dos totales del curso.
27. **Fin.**

3. PSEUDOCÓDIGO

Los arreglos usan índices desde 0. En PSeInt se utiliza un perfil que permita dimensionar con variables e índices en base 0; el número mostrado del estudiante es $i + 1$.

```
Algoritmo ControlAsistencia
    Definir estudiantes, dias, i, j Como Entero;
    Definir totalAsistencias, totalAusencias Como Entero;
    Definir asistencias, ausencias Como Entero;
    Definir marca Como Cadena;
    totalAsistencias <- 0;
    totalAusencias <- 0;
    Escribir "Numero de estudiantes:";
    Leer estudiantes;
    Mientras estudiantes <= 0 Hacer
        Escribir "Ingrese una cantidad mayor que cero:";
        Leer estudiantes;
    FinMientras
    Escribir "Numero de dias:";
    Leer dias;
    Mientras dias <= 0 Hacer
        Escribir "Ingrese una cantidad mayor que cero:";
        Leer dias;
    FinMientras
    Dimension asistencias[estudiantes], ausencias[estudiantes];
    Para i <- 0 Hasta estudiantes - 1 Hacer
        asistencias[i] <- 0;
        ausencias[i] <- 0;
        Para j <- 0 Hasta dias - 1 Hacer
            Escribir "Estudiante ", i + 1, ", dia ", j + 1;
            Escribir "Ingrese P (presente) o A (ausente):";
            Leer marca;
            Mientras marca <> "P" Y marca <> "A" Hacer
                Escribir "Marca invalida. Ingrese P o A:";
                Leer marca;
            FinMientras
            Si marca = "P" Entonces
                asistencias[i] <- asistencias[i] + 1;
                totalAsistencias <- totalAsistencias + 1;
            SiNo
                ausencias[i] <- ausencias[i] + 1;
                totalAusencias <- totalAusencias + 1;
            FinSi
        FinPara
    FinPara
    Para i <- 0 Hasta estudiantes - 1 Hacer
        Escribir "Estudiante ", i + 1;
        Escribir "Asistencias: ", asistencias[i];
        Escribir "Ausencias: ", ausencias[i];
    FinPara
    Escribir "Total de asistencias: ", totalAsistencias;
    Escribir "Total de ausencias: ", totalAusencias;
FinAlgoritmo
```

Validación y ciclo de estudiantes



5. PRUEBA DE ESCRITORIO

Cada caso es una ejecución nueva. Las marcas se ingresan por estudiante: primero todos los días del estudiante 1, luego los del estudiante 2, y así sucesivamente.

Caso 1 — Un estudiante y un día

Variable	Valor
estudiantes / días	1 / 1
marca	P
asistencias[0] / ausencias[0]	1 / 0
totalAsistencias / totalAusencias	1 / 0

Operaciones:

$i = 0$ y $j = 0$: P es válida. asistencias[0]: $0 \rightarrow 1$; totalAsistencias: $0 \rightarrow 1$.

No se registran ausencias. Ambos ciclos terminan después de una marca.

Verificación: $1 + 0 = 1$ día; total del curso: $1 \times 1 = 1$ registro.

Caso 2 — Tres estudiantes durante dos días

Variable	Valor
estudiantes / días	3 / 2
Marcas de cada estudiante	E1: P, A; E2: A, A; E3: P, P
asistencias en orden	[1, 0, 2]
ausencias en orden	[1, 2, 0]
totalAsistencias / totalAusencias	3 / 3

Operaciones:

Estudiante 1: P, A $\rightarrow$ 1 asistencia y 1 ausencia. Totales: 1 y 1.

Estudiante 2: A, A $\rightarrow$ 0 asistencias y 2 ausencias. Totales: 1 y 3.

Estudiante 3: P, P $\rightarrow$ 2 asistencias y 0 ausencias. Totales: 3 y 3.

Cada estudiante suma 2 registros. El curso suma $3 + 3 = 6 = 3 \times 2$.

Caso 3 — Cantidades y marcas inválidas

Variable	Valor
estudiantes ingresados	0 y -2 rechazados; 2 aceptado
días ingresados	0 y -1 rechazados; 2 aceptado
Marcas del estudiante 1	Día 1: X, p rechazadas; P aceptada. Día 2: A.
Marcas del estudiante 2	Día 1: A. Día 2: ? rechazada; P aceptada.
asistencias / ausencias	[1, 1] / [1, 1]
totalAsistencias / totalAusencias	2 / 2

Operaciones:

Las cantidades no positivas se rechazan antes de crear los arreglos. Se aceptan 2 estudiantes y 2 días.

Estudiante 1, día 1: X y p no son P ni A. No se avanza de día ni se incrementan contadores. Al ingresar P, asistencias[0] = 1 y totalAsistencias = 1.

Estudiante 1, día 2: A → ausencias[0] = 1 y totalAusencias = 1.

Estudiante 2, día 1: A → ausencias[1] = 1 y totalAusencias = 2.

Estudiante 2, día 2: ? se rechaza. P se acepta → asistencias[1] = 1 y totalAsistencias = 2.

Verificación: cada estudiante tiene $1 + 1 = 2$ registros; el curso tiene $2 + 2 = 4 = 2 \times 2$.

Resumen de los casos

Caso	Estudiantes	Días	Asistencias	Ausencias	Registros
1	1	1	1	0	1
2	3	2	3	3	6
3	2	2	2	2	4

Comprobación de ejecución:

Java y C++ se compilaron y ejecutaron con los tres casos. Se verificaron ambos arreglos y los totales. El tercer caso produce cuatro avisos de cantidad y tres de marca inválida, manteniendo el estudiante y día hasta recibir una marca válida.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class ControlAsistencia {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int estudiantes, dias, i, j;
        int totalAsistencias = 0, totalAusencias = 0;
        String marca;

        System.out.println("Numero de estudiantes:");
        estudiantes = scanner.nextInt();
        while (estudiantes <= 0) {
            System.out.println("Ingrese una cantidad mayor que cero:");
            estudiantes = scanner.nextInt();
        }
        System.out.println("Numero de dias:");
        dias = scanner.nextInt();
        while (dias <= 0) {
            System.out.println("Ingrese una cantidad mayor que cero:");
            dias = scanner.nextInt();
        }

        int[] asistencias = new int[estudiantes];
        int[] ausencias = new int[estudiantes];
        for (i = 0; i < estudiantes; i++) {
            asistencias[i] = 0;
            ausencias[i] = 0;
            for (j = 0; j < dias; j++) {
                System.out.println("Estudiante " + (i + 1) + ", dia " + (j + 1));
                System.out.println("Ingrese P (presente) o A (ausente):");
                marca = scanner.next();
                while (marca.equals("P") == false && marca.equals("A") == false) {
                    System.out.println("Marca invalida. Ingrese P o A:");
                    marca = scanner.next();
                }
                if (marca.equals("P")) {
                    asistencias[i]++;
                    totalAsistencias++;
                } else {
                    ausencias[i]++;
                    totalAusencias++;
                }
            }
        }

        // Mostramos los resultados cuando termina todo el registro.
        for (i = 0; i < estudiantes; i++) {
            System.out.println("Estudiante " + (i + 1));
            System.out.println("Asistencias: " + asistencias[i]);
            System.out.println("Ausencias: " + ausencias[i]);
        }
        System.out.println("Total de asistencias: " + totalAsistencias);
        System.out.println("Total de ausencias: " + totalAusencias);
        scanner.close();
    }
}
```

C++

Los arreglos se crean con el tamaño ingresado y se liberan al finalizar mediante delete[].

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int estudiantes, dias, i, j;
    int totalAsistencias = 0, totalAusencias = 0;
    string marca;

    cout << "Numero de estudiantes:" << endl;
    cin >> estudiantes;
    while (estudiantes <= 0) {
        cout << "Ingrese una cantidad mayor que cero:" << endl;
        cin >> estudiantes;
    }
    cout << "Numero de dias:" << endl;
    cin >> dias;
    while (dias <= 0) {
        cout << "Ingrese una cantidad mayor que cero:" << endl;
        cin >> dias;
    }

    int* asistencias = new int[estudiantes];
    int* ausencias = new int[estudiantes];
    for (i = 0; i < estudiantes; i++) {
        asistencias[i] = 0;
        ausencias[i] = 0;
        for (j = 0; j < dias; j++) {
            cout << "Estudiante " << (i + 1) << ", dia " << (j + 1) << endl;
            cout << "Ingrese P (presente) o A (ausente):" << endl;
            cin >> marca;
            while (marca != "P" && marca != "A") {
                cout << "Marca invalida. Ingrese P o A:" << endl;
                cin >> marca;
            }
            if (marca == "P") {
                asistencias[i]++;
                totalAsistencias++;
            } else {
                ausencias[i]++;
                totalAusencias++;
            }
        }
    }

    // Mostramos los resultados cuando termina todo el registro.
    for (i = 0; i < estudiantes; i++) {
        cout << "Estudiante " << (i + 1) << endl;
        cout << "Asistencias: " << asistencias[i] << endl;
        cout << "Ausencias: " << ausencias[i] << endl;
    }
    cout << "Total de asistencias: " << totalAsistencias << endl;
    cout << "Total de ausencias: " << totalAusencias << endl;
    delete[] asistencias;
    delete[] ausencias;
    return 0;
}
```